

PROBLEM SET

ICPC Asia EC Online Contest I

2026 · 14 Problems · 5 Hours

A	Recall
B	Mod
C	Permutation Inversions
D	Sequence
E	LCM Permutation
F	50 Years of Excellence
G	Toll Gates on a Tight Schedule
H	The First Problem
I	A Lamp of Moon
J	String
K	Wolf Game
L	Longest Common Prefix
M	Check In
N	Red Sequence

Official statements · Testing OJ

Problem A. Recall

Input file: **standard input**
Output file: **standard output**
Time limit: **2 seconds**
Memory limit: **1024 megabytes**

There is an initially empty stack S . A sequence of operations is then performed on it. There are three types of operations:

- `" + x "` means pushing the element x onto the stack.
- `" ? x "` means querying whether the element x exists in the stack. There are two possible results:
 - `" T x "` means that the element x exists.
 - `" F x "` means that the element x does not exist.
- `" - "` means popping the top element from the stack (the stack must be nonempty).

Little G recorded all `" + x "` `" T x "` and `" F x "` operations during the entire process, but did not record any `" - "` operations.

Now, Little G wants to insert some `" - "` operations into the recorded operation sequence such that:

- At all times, all elements in the stack are distinct.
- The results of all `" ? x "` queries are consistent with the operation sequence you construct.

It is guaranteed that at least one valid reconstructed sequence exists.

Input

There are multiple test cases.

The first line of the input contains an integer T ($1 \leq T \leq 10^5$), representing the number of test cases. Each test case is described as follows:

For each test case:

- The first line contains a positive integer n ($1 \leq n \leq 10^6$).
- Each of the next n lines is in the form `" + x "` `" T x "` or `" F x "` representing a recorded operation. It is guaranteed that $1 \leq x \leq 10^9$.

It is guaranteed that $1 \leq \sum n \leq 10^6$.

Output

For each test case, output one line containing a string consisting only of the characters `" + "` `" ? "` and `" - "` representing the operation sequence you reconstruct, where:

- If the i -th character is `" + "` then the i -th operation in your reconstructed sequence pushes an element onto the stack.
- If the i -th character is `" ? "` then the i -th operation in your reconstructed sequence queries whether an element exists in the stack.
- If the i -th character is `" - "` then the i -th operation in your reconstructed sequence pops the top element from the stack.

If there are multiple possible original operation sequences, you may reconstruct any one of them.

Example

standard input	standard output
2	+ - + ? - ?
4	++ ? ? ? -- + -
+ 1	
+ 1	
T 1	
F 1	
6	
+ 1	
+ 2	
T 1	
F 3	
T 2	
+ 2	

Problem B. Mod

Input file: **standard input**
Output file: **standard output**
Time limit: 6.5 seconds
Memory limit: 1024 megabytes

Give a strictly increasing sequence a of length n and a initial value v_0 .
For all $0 \leq v' \leq v_0$, find the number of permutations p of length n such that the following procedure yields final value $v = v'$:

- 1. Initialize $v \leftarrow v_0$;
- 2. For $i = 1, 2, \dots, n$, update $v \leftarrow v \bmod a_{p_i}$;

Output the answer modulo 998244353.

Input

The first line contains two integers n and v_0 ($1 \leq n < 2^{18}$, $1 \leq v_0 < 2^{18}$).
The second line contains n integers $a_1, a_2, \dots, a_n$ ($1 \leq a_1 < a_2 < \dots < a_n < 2^{18}$).

Output

Output $v_0 + 1$ integers, where the i -th integer ($1 \leq i \leq v_0 + 1$) represents the answer for $v' = i - 1$, modulo 998244353.

Examples

standard input	standard output
2 5 2 3	1 1 0 0 0 0
10 10 1 2 3 4 5 6 7 8 9 10	3628800 0 0 0 0 0 0 0 0 0 0

Problem C. Permutation Inversions

Input file: standard input
Output file: standard output
Time limit: 3 seconds
Memory limit: 1024 megabytes

There is an unknown permutation $p = (p_1, p_2, \dots, p_n)$ of the integers $1, 2, \dots, n$. You are given m pieces of information about this permutation.

Each piece of information consists of an interval $[l_i, r_i]$ and a permutation $q_{i,1}, q_{i,2}, \dots, q_{i,r_i-l_i+1}$ of the indices in this interval. It states that

$$p_{q_{i,1}} < p_{q_{i,2}} < \dots < p_{q_{i,r_i-l_i+1}}.$$

Your task is to find a permutation that satisfies all the given information and has the minimum possible number of inversions. If no such permutation exists, output -1 .

If several optimal permutations exist, output any of them.

The number of inversions of a permutation p is the number of pairs of indices (x, y) such that

$$1 \leq x < y \leq n \quad \text{and} \quad p_x > p_y.$$

Input

The first line contains a single integer T ($1 \leq T \leq 10^6$) — the number of test cases.

For each test case:

- The first line contains two integers n ($1 \leq n \leq 10^6$) and m ($1 \leq m \leq 10^6$) — the size of the permutation and the number of constraints.
- Each of the next m lines describes a constraint. The i -th constraint starts with two integers l_i and r_i ($1 \leq l_i \leq r_i \leq n$), followed by $r_i - l_i + 1$ integers $q_{i,1}, q_{i,2}, \dots, q_{i,r_i-l_i+1}$, which form a permutation of the integers from l_i to r_i .

It is guaranteed that $1 \leq \sum n \leq 10^6$, $1 \leq \sum m \leq 10^6$, and $1 \leq \sum_{i=1}^m (r_i - l_i + 1) \leq 10^6$ over all test cases.

Output

For each test case, print a single line:

- If no satisfying permutation exists, print -1 .
- Otherwise, print n space-separated integers $p_1, p_2, \dots, p_n$ representing a valid permutation that minimizes the number of inversions.

If there are multiple optimal permutations, print any of them.

Example

standard input	standard output
4	1 3 2 4
4 1	-1
2 4 3 2 4	1 3 4 2 5
3 2	2 1 4 3
1 3 1 2 3	
1 3 3 2 1	
5 2	
2 4 4 2 3	
2 4 4 2 3	
4 2	
2 2 2	
1 4 2 1 4 3	

Problem D. Sequence

Input file: **standard input**
Output file: **standard output**
Time limit: 1 second
Memory limit: 1024 megabytes

Given a 01-sequence $s_1, s_2, \dots, s_n$ of length n , define

$$p_i = \sum_{j=1}^i [s_i \neq s_j]$$

where $[]$ is the Iverson bracket (1 if the condition holds, 0 otherwise).

You are given the integer n and the multiset $\{p'_1, p'_2, \dots, p'_n\}$ of the values $\{p_1, p_2, \dots, p_n\}$.

Count the number of sequences s that satisfy the condition.

Output the answer modulo 998244353.

It is guaranteed that there exists at least one string satisfying the conditions.

Input

The first line contains a single integer n ($1 \leq n \leq 10^5$).

The second line contains n integers $p'_1, p'_2, \dots, p'_n$ ($0 \leq p'_i \leq n - 1$) — the multiset.

Output

Print a single integer — the number of valid binary strings modulo 998 244 353.

Examples

standard input	standard output
1 0	2
3 0 1 1	4

Problem E. LCM Permutation

Input file: standard input
Output file: standard output
Time limit: 1 second
Memory limit: 1024 megabytes

Construct a permutation $p_1, p_2, \dots, p_n$ of length n that satisfies $\text{lcm}_{1 \leq i \leq n} (p_i + p_{(i \bmod n)+1}) \leq 20n$.

Input

The first line contains a positive integer t ($1 \leq t \leq 10^4$), representing the number of test cases. Each test case contains a single integer n ($1 \leq n \leq 8 \cdot 10^6$), representing the length of the permutation. It is guaranteed that the sum of n over all test cases does not exceed $8 \cdot 10^6$.

Output

For each test case, output the permutation $p_1, p_2, \dots, p_n$ that satisfies the condition.

It is guaranteed that at least one such permutation exists.

Example

standard input	standard output
1 3	1 2 3

Note

In the example, $\text{lcm}(p_1 + p_2, p_2 + p_3, p_3 + p_1) = \text{lcm}(3, 5, 4) = 60 \leq 20n$.

You can use the following code to speed up the output.

```
#include <bits/stdc++.h>
using namespace std;

int ptr = 0; char buf[1 << 23];
inline void writeChar(char c) {
    if (ptr == 1 << 23) fwrite(buf, 1, ptr, stdout), ptr = 0; buf[ptr++] = c;
}
inline void writeInt(int x) {
    int t = 0; char s[10];
    while (x) s[t++] = (char)('0' + x % 10), x /= 10;
    while (t) writeChar(s[--t]);
}
inline void flushOut() {
    if (ptr) fwrite(buf, 1, ptr, stdout), ptr = 0;
}

int main() {
    // Your code here...
    flushOut();
    return 0;
}
```


Problem F. 50 Years of Excellence

Input file: standard input
Output file: standard output
Time limit: 1 second
Memory limit: 1024 megabytes

The International Collegiate Programming Contest has inspired generations of students throughout fifty years of excellence.

Consider n consecutive years. In each year, exactly m problems are prepared, and every problem has an integer rating.

The score of a year is the sum of the ratings of all m problems in that year. The score immediately before the first year is defined to be 0.

A year is called *excellent* if its score is strictly less than the score of the previous year. This definition also applies to the first year, whose score is compared with 0.

Determine how many years are excellent.

Input

The first line contains two integers n and m ($1 \leq n \leq 50$, $1 \leq m \leq 13$)—the number of years and the number of problems in each year. Each of the next n lines contains m integers $a_{i,1}, a_{i,2}, \dots, a_{i,m}$ ($-1000 \leq a_{i,j} < 0$)—the ratings of the problems in year i .

Output

Print one integer—the number of excellent years.

Examples

standard input	standard output
5 3 -100 -200 -300 -100 -200 -250 -150 -200 -200 -100 -150 -250 -200 -200 -250	2
4 2 -5 -5 -6 -5 -10 -1 -6 -6	3

Note

In the first example, the scores of the five years are -600 , -550 , -550 , -500 , and -650 . The first year is excellent because $-600 < 0$, and the fifth year is excellent because $-650 < -500$. Equality is not sufficient, so the third year is not excellent.

In the second example, the scores are -10 , -11 , -11 , and -12 . The first, second, and fourth years are excellent.

Problem G. Toll Gates on a Tight Schedule

Input file: standard input
Output file: standard output
Time limit: 4 seconds
Memory limit: 1024 megabytes

You have to cross the city before the last train leaves.

The city has n junctions and m one-way roads. Each road takes exactly one minute to pass. Junction i has a toll gate, and passing through it costs c_i coins.

You start at junction 1 and want to reach junction n . Since you are in a hurry, you will only consider routes that use the minimum possible number of roads.

Among all such shortest routes, count how many have total toll at most C . The total toll of a route is the sum of c_i over all junctions on the route, including junctions 1 and n .

Because the answer may be large, output it modulo 2^{64} .

Input

The first line contains three integers n , m , and C ($1 \leq n \leq 90$, $0 \leq m \leq 300$, $0 \leq C \leq 10^{12}$).

The second line contains n integers $c_1, c_2, \dots, c_n$ ($0 \leq c_i \leq 10^{12}$), where c_i is the toll of junction i .

Each of the next m lines contains two integers u and v ($1 \leq u, v \leq n$, $u \neq v$), denoting a one-way road from u to v .

There are no repeated directed roads.

Output

Print one integer: the number of shortest routes from junction 1 to junction n whose total toll is at most C , modulo 2^{64} .

Example

standard input	standard output
4 5 8 1 2 3 4 1 2 1 3 2 4 3 4 2 3	2

Note

In the example, the shortest routes from junction 1 to junction 4 use exactly two roads.

They are $1 \rightarrow 2 \rightarrow 4$, with total toll $1 + 2 + 4 = 7$, and $1 \rightarrow 3 \rightarrow 4$, with total toll $1 + 3 + 4 = 8$.

Both routes have total toll at most 8.

The route $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$ is not counted, because it is not a shortest route.

Problem H. The First Problem

Input file: **standard input**
Output file: **standard output**
Time limit: 1 second
Memory limit: 1024 megabytes

For a string A , let $pref(A, i)$ denote the prefix of A of length i .
For strings A, B , let $occur(A, B)$ denote the number of occurrences of A in B .
Define:

$$score(A, B) = \sum_{i=1}^{len(A)} \sum_{j=1}^{len(B)} occur(pref(A, i), pref(B, j))$$

That is, the sum over every prefix of A of its number of occurrences in every prefix of B .
You are given a text string T .
There are q queries, each given by a_i pattern strings $S_1, S_2, \dots, S_{a_i}$ and an integer b_i .
For each query, find the minimum x such that:

$$\sum_{j=1}^{a_i} score(S_j, pref(T, x)) \geq b_i$$

If no such x exists, output -1 .

Input

The first line contains the text string $\mathbf{T}$ ($1 \leq |T| \leq 2 \times 10^5$, where $|T|$ denotes the length of T).
The second line contains a single integer $\mathbf{q}$ ($1 \leq q \leq 2 \times 10^5$).
For each query, the first line contains two integers $\mathbf{a_i}, \mathbf{b_i}$ ($1 \leq a_i \leq 2 \times 10^5, 1 \leq b_i \leq 10^{18}$). The following $\mathbf{a_i}$ lines each contain a pattern string $\mathbf{S_1}, \mathbf{S_2}, \dots, \mathbf{S_{a_i}}$ in order.
 $\sum |S| \leq 2 \times 10^5$, where $\sum |S|$ is the total length of all pattern strings across all queries in one test case.
All strings consist of lowercase English letters.

Output

Output q lines, the answer to each query in order.

Example

standard input	standard output
abac	3
3	4
1 5	-1
aba	
2 10	
a	
ac	
1 1	
x	

Problem I. A Lamp of Moon

Input file: **standard input**
Output file: **standard output**
Time limit: **3 seconds**
Memory limit: **1024 megabytes**

Night unfolds across rivers and mountains, and moonlight falls upon the water like a lamp gently lifted in someone's hands.

Long ago, stargazers traced a vast lattice across the heavens. Every star was given its own position (i, j) , yet even stars that lie close together do not necessarily shine with the same light.

A star at (i, j) may resonate with certain ancient celestial tracks. Whenever a prime divides both i and j , the star is said to be in harmony with one such track. Beyond these resonances, every star also carries a glimmer of its own, one that will never fade. Thus, its radiance is $\omega(\gcd(i, j)) + 1$, where $\omega(x)$ denotes the number of distinct prime factors of x .

No one's eyes can hold the entire night sky in a single glance. Each time you look up, you can bear at most L units of starlight. You begin with the lowest line of sight, at the star $(n, 1)$, and slowly raise your gaze, encountering the stars in increasing order of their slope j/i . If several stars overlap along the same line of sight, the nearer one (i.e. the one with smaller i) is encountered first. You gather their light one by one, until the next star can no longer be seen in its entirety.

The night is vast and the stars are countless, yet gaze must finally come to rest somewhere. The last star that can be seen fully is the one I hold especially dear.

To the eye, it may be only a distant point of light. To me, however, it calls to mind someone who has left the familiar eaves behind and set out along a road of her own. She is not a destination waiting to be reached, but a traveler already on her way.

She has gone so far that the night has blurred her figure. Other stars lie along the same line of sight, overlapping in the distant sky, so I can no longer tell exactly where she is. All I can do is keep looking in her direction.

So I ask only this of you: tell me which direction I should look, namely the slope j/i of the star.

Formal Description

For a positive integer x , let $\omega(x)$ denote the number of distinct prime factors of x . In particular, $\omega(1) = 0$.

You are given a positive integer n . Consider all integer points (i, j) satisfying $1 \leq i, j \leq n$. The weight of (i, j) is $a_{i,j} = \omega(\gcd(i, j)) + 1$.

Sort these n^2 points by j/i in increasing order. If two points have the same value of j/i , the point with smaller i comes first. Denote the resulting sequence by $c_1, c_2, \dots, c_{n^2}$.

Let $c_k = (i_k, j_k)$ and define the prefix weight sum $S_k = \sum_{r=1}^k a_{i_r, j_r}$.

For each query, you are given a positive integer L . Let k be the largest positive integer satisfying $S_k \leq L$. If the total weight of all points is at most L , let $k = n^2$.

Output the slope j_k/i_k of c_k . More precisely, output two coprime positive integers p, q such that $p/q = j_k/i_k$.

It can be proved that the answer exists and is unique for every query.

Input

The first line contains two positive integers n ($1 \leq n \leq 10^9$) and T ($1 \leq T \leq 5$).

The following T lines each contain one positive integer L ($1 \leq L \leq 2 \times 10^{18}$).

Output

For each query, output one line containing two coprime positive integers p, q , representing the answer slope p/q .

Examples

standard input	standard output
2 3 1 3 100	1 2 1 1 2 1
30 5 1 1288 1289 1290 2000000000000000000	1 30 29 1 30 1 30 1 30 1
100000000 5 7261237189656552 7261237189656553 7261237189656554 7188624601740119 7261241259305175	1 1 1 1 100000000 99999999 99 100 99999989 99999931

Note

$1 \leq n \leq 10^9, 1 \leq T \leq 5, 1 \leq L \leq 2 \times 10^{18}$.

Perhaps each of us is walking a road like hers.

At times, this mountain path is paved with disappointments known to no one and joys that last only for a moment. To keep moving forward is not to sever ourselves from the past, nor to abandon who we are today. The roads on which we lost our way, the nights during which we came to a halt, and the people who once walked beside us will all become part of the landscape, carried with us toward places farther away.

The road ahead may be long and difficult. There will still be nights when you cannot see the way, and moments when you wonder whether you can continue. Yet I hope you will not mistake a distant destination for proof that all your steps have been in vain. To have come this far is itself to have become a shining star; and those who are willing to look up once more will always find a new direction among the constellations.

I offer you a lamp of moon, wishing all of you fair winds, however distant the mountains and long the road ahead.

Problem J. String

Input file: **standard input**
Output file: **standard output**
Time limit: 3 seconds
Memory limit: 1024 megabytes

You are given a string S of length n , indexed from 1 to n . Every character of S is either N or H.
You must process T operations in order. An operation is described by three integers op , p , and k , and consists of repeating the following procedure exactly k times. Every repetition starts at position p and uses the current string obtained after all previous repetitions and operations.
For one repetition, choose a direction and a stopping character according to op :

- If $op = 1$, move to the right and use N as the stopping character.
- If $op = 2$, move to the right and use H as the stopping character.
- If $op = 3$, move to the left and use N as the stopping character.
- If $op = 4$, move to the left and use H as the stopping character.

Start at position p . At every visited position, first flip its character: N becomes H, and H becomes N. If the character before the flip was the stopping character, the repetition ends immediately. Otherwise, move by one position in the chosen direction and continue. If there is no next position in that direction, the repetition ends after flipping the boundary character.
Determine the string after all operations have been performed.

Input

The first line contains two integers n and T ($1 \leq n, T \leq 10^5$) — the length of the string and the number of operations.
The second line contains a string S of length n . Every character of S is either N or H.
Each of the next T lines contains three integers op_i , p_i , and k_i ($1 \leq op_i \leq 4$, $1 \leq p_i \leq n$, $0 \leq k_i \leq 10^{18}$), describing the i -th operation.

Output

Output one string of length n — the final value of S after all T operations.

Example

standard input	standard output
5 5 HHNNN 1 5 0 4 1 3 1 5 1 3 1 3 4 4 3	HNNHH

Problem K. Wolf Game

Input file: standard input
Output file: standard output
Time limit: 2 seconds
Memory limit: 1024 megabytes

There are n people standing in a row from left to right. Each person is either a villager or a werewolf. Villagers always tell the truth. Werewolves may tell either the truth or a lie. It is also known that no two adjacent people are both werewolves.

The statement of the i -th person is described by a pair (d_i, b_i) :

- if $d_i = \text{L}$, the person says that there are exactly b_i werewolves to their left;
- if $d_i = \text{R}$, the person says that there are exactly b_i werewolves to their right.

For each test case, construct any possible assignment of people to villagers and werewolves, or determine that no such assignment exists.

Input

The first line contains an integer T ($1 \leq T \leq 10^5$), the number of test cases. Each test case consists of three lines. The first line contains an integer n ($2 \leq n \leq 5 \cdot 10^5$). The second line contains a string d of length n , consisting only of characters **L** and **R**. The third line contains n integers $b_1, b_2, \dots, b_n$ ($0 \leq b_i \leq n$). The sum of n over all test cases does not exceed $5 \cdot 10^5$.

Output

For each test case, if there is no valid assignment, print -1. Otherwise print a binary sequence of length n . The i -th value must be 0 if the i -th person is a villager, and 1 if the i -th person is a werewolf. You may print it either as one binary string or as n space-separated integers.

Example

standard input	standard output
3	001
3	-1
RLL	10101
1 0 0	
3	
RRR	
2 0 1	
5	
RRLRR	
1 2 1 1 0	

Problem L. Longest Common Prefix

Input file: **standard input**
Output file: **standard output**
Time limit: **1 second**
Memory limit: **1024 megabytes**

You are given n lowercase English strings $s_1, s_2, \dots, s_n$.

For each pair of integers (i, j) satisfying $1 \leq j \leq i \leq n$, define $f_{i,j}$ as the maximum possible length of the longest common prefix of exactly j strings selected from the first i strings. Formally,

$$f_{i,j} = \max_{T \subseteq \{1, 2, \dots, i\}, |T|=j} |\text{LCP}(s_k \mid k \in T)|.$$

The longest common prefix of a single string is the string itself.

For each $i = 1, 2, \dots, n$, calculate

$$\sum_{j=1}^i (f_{i,j} \oplus j),$$

where $\oplus$ denotes the bitwise XOR operation.

Input

The first line contains an integer n ($1 \leq n \leq 5 \cdot 10^5$), the number of strings.

Each of the next n lines contains a non-empty string s_i consisting only of lowercase English letters.

It is guaranteed that

$$\sum_{i=1}^n |s_i| \leq 5 \cdot 10^5.$$

Output

Print n lines.

The i -th line should contain one integer:

$$\sum_{j=1}^i (f_{i,j} \oplus j).$$

Examples

standard input	standard output
4 a ab abc abcd	0 6 4 12
8 ab b ba ab aa ba a bb	3 5 9 10 14 20 28 36

Note

The longest common prefix of several strings is the longest string that is a prefix of every selected string.

The symbol $\oplus$ denotes the bitwise XOR operation.

Problem M. Check In

Input file: **standard input**
Output file: **standard output**
Time limit: **1 second**
Memory limit: **1024 megabytes**

In an ICPC contest, every participating team must first check in at the registration desk. You are asked to maintain the information system used for the sign-in process.

There is a roster containing the names of n teams. All team names are pairwise distinct, and every name consists only of uppercase or lowercase English letters.

After the roster, there are m queries. Each query gives a name (which also consists only of uppercase or lowercase English letters). For each query, you must classify it into exactly one of the following three outcomes:

- ****Successful sign-in**** — the name belongs to a valid team, and that team has not signed in before.
- ****Not a valid team**** — the name does not belong to any team in the roster.
- ****Already signed in**** — the name belongs to a valid team, but that team has already signed in earlier.

Input

The first line contains two integers n and m ($1 \leq n, m \leq 10^5$) — the number of teams in the roster and the number of queries.

The next n lines each contain a team name. All team names are distinct.

The next m lines each contain a query name.

Every name (both roster names and query names) consists only of uppercase or lowercase English letters. The total number of characters over all names (roster names and query names combined) does not exceed 10^6 .

Output

For each of the m queries, print one line containing exactly one of the following words:

- **‘OK’** — if the query results in a successful sign-in;
- **‘WRONG’** — if the name is not a valid team;
- **‘REPEAT’** — if the name has already signed in.

Example

standard input	standard output
5 9	OK
RebornAsaVegetableDog	OK
LetItRot	OK
NaiveBirds	OK
OpenYourBrain	OK
CaptainTakesMeNot	REPEAT
CaptainTakesMeNot	WRONG
OpenYourBrain	WRONG
NaiveBirds	REPEAT
LetItRot	
RebornAsaVegetableDog	
NaiveBirds	
OneStepAway	
Metempsychosis	
CaptainTakesMeNot	

Problem N. Red Sequence

Input file: **standard input**
Output file: **standard output**
Time limit: 2 seconds
Memory limit: 1024 megabytes

You are given a sequence of length n .
For each position i , there are between 1 and 3 chess pieces. The chess pieces have three possible colors: red, yellow, and blue.
It is guaranteed that there are no two chess pieces of the same color at the same position.
You need to divide the sequence into several continuous segments.
A continuous segment is called red if the number of red chess pieces in this segment is not less than the number of yellow chess pieces and is also not less than the number of blue chess pieces.
More formally, for a segment:

$$\text{red count} \geq \max(\text{yellow count}, \text{blue count})$$

Your task is to maximize the sum of lengths of all red segments after the division.
Output this maximum possible sum.

Input

The first line contains an integer n ($1 \leq n \leq 1000000$).
The following n lines describe the sequence.
The i -th line contains three integers r_i , y_i , and b_i ($0 \leq r_i, y_i, b_i \leq 1$), where:
 $r_i = 1$ means that the i -th position contains a red chess piece, $y_i = 1$ means that it contains a yellow chess piece, and $b_i = 1$ means that it contains a blue chess piece.
It is guaranteed that $1 \leq r_i + y_i + b_i \leq 3$ for every i .

Output

Print one integer: the maximum possible sum of lengths of all red segments.

Example

standard input	standard output
10 1 0 0 0 1 1 0 1 1 1 1 0 0 1 1 1 0 1 0 1 0 0 1 1 1 1 1 1 0 1	7